

Ortelius Workshop Tool

Project Specification

Final Agreed Version — Anthony & Mohand

May 21, 2026

What We're Building

A workshop facilitation tool where a consultant runs a session, participants submit ideas live, AI clusters and summarises them, and a PDF report is generated. Built for Ortelius consultants to use in real workshops.

The Flow

1. Facilitator creates a session (picks topic + framework)
2. Participants join with just a name — no accounts
3. Participants submit ideas in real-time
4. Facilitator triggers AI analysis
5. Claude groups ideas into SWOT/PESTEL categories
6. PDF report is generated for download

Architecture

Flutter app --realtime--> Firebase Firestore (ideas, votes)
Flutter app --REST-----> FastAPI backend (sessions, analysis, PDF)
FastAPI backend --reads----> Firebase Firestore (ideas for analysis)
FastAPI backend --calls----> Claude API (clustering + summarisation)
FastAPI backend --stores---> PostgreSQL / Supabase (sessions, analysis, reports)

Why hybrid (Firestore + PostgreSQL): Firestore handles real-time — ideas, votes, live updates during the workshop. PostgreSQL handles persistence — sessions, analysis results, PDF reports, structured queries. Firestore charges per read/write, PostgreSQL is fixed cost.

Responsibilities

Layer	What it does
Flutter	UI, real-time idea sync, voting, QR code generation
Firebase Firestore	Live workshop data (ideas, votes, participants)
Firebase Auth	Anonymous auth — participants don't create accounts
FastAPI	Session management, AI analysis, PDF generation
Claude API	Idea clustering + summarisation
PostgreSQL (Supabase)	Sessions, analysis results, reports
ReportLab	PDF generation

Tech Stack

Area	Choice
Frontend	Flutter (web + mobile)
State management	Riverpod
Real-time database	Firebase Firestore
Backend	FastAPI (Python)
AI	Anthropic Claude API
PDF	ReportLab
Auth	Firebase Anonymous Auth
Persistence	PostgreSQL (Supabase)
Hosting	Firebase Hosting + Render/Railway (or Azure)

API Endpoints

Sessions

Method	Path	What it does
POST	/sessions	Creates a workshop session
GET	/sessions/{id}	Gets session details
POST	/sessions/{id}/join?name=...	Participant joins with name (query param)

Ideas

Method	Path	What it does
POST	/sessions/{id}/ideas	Submit an idea (testing only)
GET	/sessions/{id}/ideas	List all ideas

In production, Flutter writes ideas directly to Firestore. The POST endpoint is for backend testing.

Analysis

Method	Path	What it does
POST	/sessions/{id}/analyse	Runs AI analysis (synchronous — returns result directly)
GET	/sessions/{id}/analysis	Fetches stored analysis (refresh without re-running)
GET	/sessions/{id}/report	Downloads PDF report

Utility

Method	Path	What it does
GET	/health	Confirms backend is running
GET	/docs	Swagger UI — test everything in browser

Data Models

Session

```
{
  "id": "uuid",
  "topic": "Improve onboarding",
  "framework": "swot",
  "access_code": "ABCD12",
  "status": "active",
  "created_at": "ISO 8601",
}
```

Participant

```
{
  "id": "uuid",
  "name": "Anna",
  "joined_at": "ISO 8601",
}
```

Idea

```
{
  "id": "uuid",
  "session_id": "uuid",
  "participant_id": "uuid",
  "participant_name": "Anna",
  "category": "strength" (optional),
  "content": "Better documentation for new hires",
  "votes": 3,
  "created_at": "ISO 8601",
}
```

Category field on ideas is optional — participants can pre-tag ideas, but AI will categorise everything during analysis regardless.

Analysis Result

```
{
  "session_id": "uuid",
  "framework": "swot",
  "categories": { strengths: [...], weaknesses: [...], opportunities: [...], threats: [...] },
  "key_themes": ["string"],
  "decisions_made": ["string"],
  "open_questions": ["string"],
  "recommended_next_steps": ["string"],
}
```

Every clustered idea references the original idea_id — provenance requirement from Ortelius. The PDF shows who said what.

Firestore Structure

```
sessions/{session_id}/
+-- topic: string
+-- framework: string
+-- status: string
+-- participants: array
sessions/{session_id}/ideas/{idea_id}
+-- participant_id: string
+-- participant_name: string
+-- category: string (optional)
+-- content: string
+-- votes: number
+-- created_at: timestamp
```

Firestore Security Rules

- Users can only edit their own ideas
- Authenticated anonymous users only
- Analysis writable only by backend

What Flutter Calls

```
Flutter -> POST /sessions (create workshop)
Flutter -> POST /sessions/{id}/join (participant joins)
Flutter -> Firestore (real-time idea input + voting)
Flutter -> POST /sessions/{id}/analyse (trigger AI)
Flutter -> GET /sessions/{id}/analysis (fetch results)
Flutter -> GET /sessions/{id}/report (download PDF)
```

Flutter generates QR codes locally — it's just encoding a URL, no backend needed.

Repo Structure

```
workshopstodverktyg/
+-- backend/ <-- FastAPI (Anthony)
| +-- app/
| +-- main.py
| +-- config/
| +-- models/
| +-- routes/
| +-- services/
| | +-- claude_service.py
| | +-- firestore_service.py
```

```
| | +-- pdf_service.py
| | +-- analysis_service.py
| +-- prompts/
+-- frontend_flutter/ <-- Flutter (Mohand)
| +-- lib/
| +-- main.dart
| +-- core/
| +-- models/
| +-- services/
| +-- features/
| | +-- session/
| | +-- workshop/
| | +-- analysis/
| | +-- report/
| +-- widgets/
+-- firebase/
+-- docs/
+-- README.md
```

Development Phases

Phase 1 — Workshop Setup

Weeks 1–2

- Create session
- Join session with name
- Access code generation
- QR code generation (frontend)
- Basic real-time dashboard

Phase 2 — Realtime Workshop

Weeks 3–4

- Submit ideas to Firestore
- Real-time dashboard updates
- Upvote ideas
- Facilitator dashboard

Phase 3 — AI Analysis

Weeks 5–6

- Claude integration
- SWOT/PESTEL clustering
- Structured JSON output
- Prompt design + evaluation

Phase 4 — Reports

Weeks 7–8

- PDF generation (ReportLab)
- Consultant-ready template
- Download flow

Phase 5 — Testing & Polish

Weeks 9–10

- Multi-participant testing
- Real-time sync stress tests
- Error handling
- User testing with Ortelius consultants

Phase 6 — Delivery

Weeks 11–12

- Cloud deployment

- Documentation
- Demo prep
- Final presentation

Claude Prompt Strategy

Keep it simple. No agents, no LangChain, no RAG.

```
You are analysing workshop ideas.  
Group the following ideas into SWOT categories.  
Return ONLY valid JSON.  
Ideas:  
{ideas}
```

Prompt versions tracked in prompts/ directory. Each version logged with what changed and accuracy results.

Success Criteria

- Workshop setup < 2 minutes
- Mobile participation works without install
- Real-time updates feel instant
- AI groups ideas correctly (≥80% accuracy on eval dataset)
- PDF generated < 30 seconds
- Consultants can demo it

Error Handling

All errors return {"detail": "error message"} with HTTP status code:

Status	Meaning
200	Success
404	Session or resource not found
422	Validation error
500	Server error

Philosophy

Optimise for:

- Simplicity
- Maintainability
- Demoability
- Clear documentation

NOT for:

- Enterprise scalability
- Microservices
- Complex infrastructure

The cleanest MVP is the best MVP.